

Computer Vision

Assignmen1: Image Sensing Pipeline

Student ID : 313706018

Name : 陳昱凱

a. Discuss different treatments of different Bayer patterns when: i. applying white balance mask into original image.

The core objective of white balance is to adjust image colors to accurately reflect the lighting conditions. Since each pixel in a Bayer array sensor captures only a single color (red, green, or blue), image processing systems must **adjust the gain of each color channel based on the characteristics of the light source to achieve proper color balance.**

A typical Bayer array consists of approximately 50% green pixels and 25% red and blue pixels. This design enhances brightness representation, as the human eye is most sensitive to luminance variations in green light. When light enters the sensor, it first passes through a color filter layer, allowing each pixel to capture only a specific color. **Common Bayer patterns include BGGR, RGGB, GBRG, and GRBG,** with different arrangements affecting color reconstruction and white balance calculations.

White balance adjustment is crucial under varying color temperatures. For example, in warm lighting conditions (such as incandescent bulbs), images may appear overly yellow, prompting the system to increase the gain of the blue channel. Since different Bayer patterns determine the spatial distribution of color pixels, the method for white balance adjustment must be adapted accordingly. Differences in red and blue pixel arrangements between BGGR and RGGB affect white balance calculations, while variations in green pixel placement between GBRG and GRBG require corresponding compensation strategies to ensure accurate color reproduction.

a. Discuss different treatments of different Bayer patterns when:

ii. doing mosaic algorithm.

When implementing a mosaic algorithm, different Bayer patterns require consideration of their unique arrangements of red (R), green (G), and blue (B) filters. The Bayer filter array is a color filter pattern used in digital image sensors, where each pixel captures only one of the three colors—red, green, or blue—depending on its position within the Bayer pattern. **The goal of the mosaic algorithm is to simulate the raw sensor output by retaining only the corresponding single-channel value and discarding the others.**

Different Bayer patterns (such as RGGB, BGGR, GBRG, and GRBG) influence how color information is mapped during the mosaic process. Each pattern follows a distinct 2×2 color arrangement, meaning the rule for determining which color to retain at each pixel varies depending on the pattern. The main differences lie in the placement of red and blue pixels, as well as the distribution of green pixels. Since the human eye is more sensitive to green, green pixels are typically more frequent than red and blue ones.

During the mosaic transformation, the input image is usually in RGB format, while **the output is a single-channel image, where each pixel retains only the corresponding Bayer filter value.** This process involves mapping each pixel position to the correct color channel according to the Bayer pattern. For example, in the RGGB pattern, the top-left pixel corresponds to red (R), the adjacent pixel to the right corresponds to green (G), the pixel below remains green (G), and the bottom-right pixel corresponds to blue (B).

- a. Discuss different treatments of different Bayer patterns when:
- ii. doing mosaic algorithm.

```
def mosaic(img, pattern):  
    ...  
    Input:  
    |   img: H*W*3 numpy array, input image.  
    |   pattern: string, 4 different Bayer patterns (GRBG, RGGB, GBRG, BGGR)  
    Output:  
    |   output: H*W numpy array, output image after mosaic.  
    ...  
  
    # Get image shape  
    H, W, _ = img.shape  
    output = np.zeros((H, W))  
  
    # Bayer mask 2x2  
    pattern_dict = {  
        'RGGB': [[0, 1], [1, 2]], # R, G; G, B  
        'BGGR': [[2, 1], [1, 0]], # B, G; G, R  
        'GRBG': [[1, 0], [2, 1]], # G, R; B, G  
        'GBRG': [[1, 2], [0, 1]] # G, B; R, G  
    }  
  
    # Check pattern  
    if pattern not in pattern_dict:  
        raise ValueError("Unsupported Bayer pattern.")  
  
    # Assign pixel values  
    channels = pattern_dict[pattern]  
  
    for i in range(H):  
        for j in range(W):  
            channel = channels[i % 2][j % 2]  
            output[i, j] = img[i, j, channel]  
  
    return output
```

```
def demosaic(img, pattern):  
    ...  
    Input:  
    |   img: H*W numpy array, input RAW image.  
    |   pattern: string, 4 different Bayer patterns (GRBG, RGGB, GBRG, BGGR)  
    Output:  
    |   output: H*W*3 numpy array, output de-mosaic image.  
    ...  
  
    ##### Using Python colour_demosaicing library  
    ##### You can write your own version, too  
    output = demosaicing_CFA_Bayer_Malvar2004(img, pattern)  
    output = np.clip(output, 0, 1)  
  
    return output
```

b. Show the image results of each step as p.6/7 in HW1.pdf.

Inverse ISP

Original Image



Image after inverse tone



Image after CIE



Image after CCM



Image after mosaic



Image after Inverse WB



ISP

image Raw



Image after WG



Image after demosaic



Image after Color Correction



Image after XYZ2RGB



Image after tone mapping

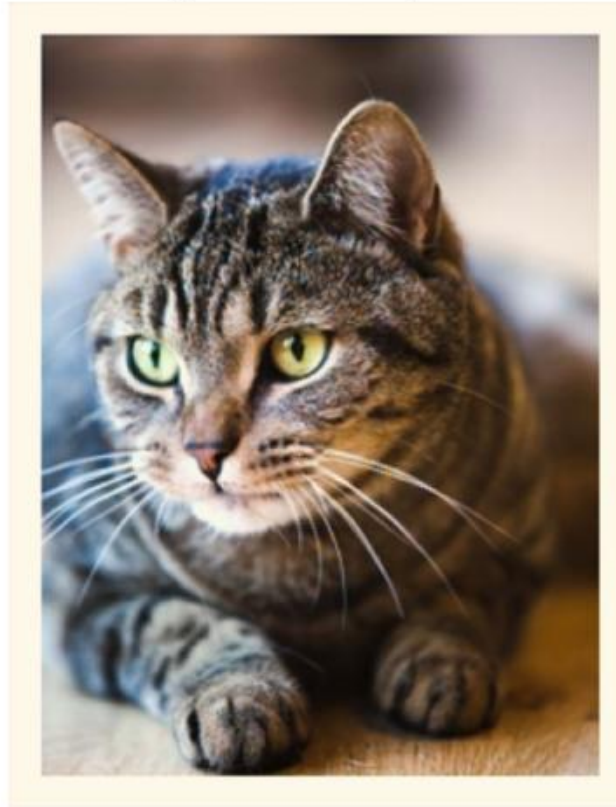


c. Show the image results of inverse ISP and ISP as p.19 in HW1.pdf. Additionally, compare the performance results of this task using PSNR.

Original Image



Image after inverse process



```
def calculate_psnr(img1, img2):  
    ...  
    Input:  
    |   img1, img2: H*W*3 numpy array  
    Output:  
    |   psnr: the peak signal-to-noise ratio value  
    ...  
    # Calculate the mean square error  
    mse = np.mean((img1 - img2) ** 2)  
  
    # Assume the image is in 0~255  
    max_value = 255.0  
  
    # if two images are the same, return inf  
    if mse == 0:  
        return float('inf')  
  
    # Calculate PSNR  
    return 20 * np.log10(max_value / (np.sqrt(mse)))  
[36] ✓ 0.0s  
  
print(f"PSNR: {calculate_psnr(img, img_tm):.2f} db")  
[43] ✓ 0.0s  
... PSNR: 80.04 db
```

d. In recent AI de-noising methods, in order to generate paired data for training, we will add synthetic noise to clean image on RAW domain instead of RGB domain. Explain the reason.

This is because **RAW data records the sensor's original signals and noise**, directly reflecting the physical conditions during image capture. When RAW data undergoes the Image Signal Processing (ISP) pipeline, it experiences a series of operations—such as demosaicing, white balancing, color correction—that not only enhance visual quality but also alter or even partially suppress the original noise distribution and characteristics due to **nonlinear processing**.

Therefore, directly adding noise in the RGB domain may fail to accurately simulate the sensor's original noise, leading to a model that performs sub optimally when dealing with real-world noise. In contrast, adding noise in the RAW domain preserves the intrinsic noise characteristics captured by the sensor. **After the ISP processing, the noise pattern in the resulting image is closer to the real situation**, which helps the model learn an effective noise removal strategy.